ColaZZang → KullChip
가족의 산책 기록에서 센서 분석으로

같은 반려견을 가족이 번갈아 산책시키며 시작했습니다. 앞선 산책의 거리·시간·배변 위치를 알아야 다음 산책을 조절하기 쉬웠습니다. ColaZZang은 휴대폰·워치 기록 앱, KullChip은 센서 자동 기록을 시도한 후속 프로젝트입니다.

ColaZZang: 워치 입력 → 모바일의 활성 산책·GPS·Room. KullChip 후기: 워치 RAW·spool → 시간축 정렬 → 모바일 Room 저장 → ACK → 원본 정리. 자동 행동·전조 판정은 최종 제품 기준에 못 미쳐 중단했습니다.

1. 워치는 입력하고, 모바일이 산책과 위치를 연결

리드줄을 잡은 채 휴대폰을 꺼내는 불편을 워치의 마커 입력으로 줄이려 했습니다. 워치 Lap 입력을 활성 산책·최근 GPS와 연결해 저장하는 코드에서 명령 수신과 데이터 소유 책임을 볼 수 있습니다. ColaZZang 원본은 app·wear·domain·data·sync·stopwatch·style의 7개 모듈로 나눴습니다.

2. 전송 완료와 저장 완료는 달랐다

센서를 보냈다는 응답만으로 워치 원본을 지우면 휴대폰 저장 실패 때 복구할 수 없습니다. RAW 묶음의 durable 저장 후 ACK 생성 순서로 구성했습니다. 워치는 확인받지 못한 묶음을 보관·재전송하고, 모바일은 batch ID·데이터 내용·기기 세션·순번으로 중복과 누락을 검사합니다.

늦게 온 센서를 수신 시각으로 라벨에 붙이면 다른 행동을 학습할 수 있습니다. 요청 identity·echo·왕복 중간 시각을 확인하는 시계 정렬로 워치·폰 offset을 구하고 세션별 원본 시각과 정렬 시간축을 구분했습니다. 장기 drift·모든 단절에서 무손실을 입증한 결과까지 의미하지는 않습니다.

3. ML0·ML1·ML2를 서로 다른 책임으로

ML0는 학습 모델이 아니라 센서 좌표 정규화입니다. quaternion·heading이 충분한 변환과 중력 기준 fallback을 나누고 정렬 품질·RAW hash를 남겼습니다. 센서가 어느 방향을 실제로 관측했는지 모르는 상태를 숨기지 않으려는 설계입니다.

ML1은 고정 공통 backbone과 이동·자세·머리·코·입 등 6축 head를 사용하고 작은 head를 개인화합니다. ML2는 몸짓 확률·불확실성의 시간 흐름을 읽습니다. 축·라벨·상위 모델 SHA를 검사하고 입력 hash를 만드는 코드로 다른 모델 버전의 결과가 섞이지 않게 했습니다. 독립 산책 holdout·checkpoint 복원·후보 승격도 제품 적용과 나눴습니다.

4. 근거 설명과 여러 서비스의 공통 계약

이전 android 세대는 Rust/JNI·LanceDB 기록·벡터 검색, 원본·추론·모델·사용자 수정을 잇는 온톨로지/XAI를 실험했습니다. XAI가 인용한 지표·근거 ID의 존재 검사를 두었습니다. 이 검사는 인과관계나 문장 진실성을 보증하는 것이 아닙니다. 후기 android-next의 Room RAW-first 구조와는 별도 세대입니다.

과거 서버는 공통·도메인 27개 crate와 실행 서비스 5개로 나눴습니다. 공통 Docker 빌드 뒤 선택한 한 바이너리만 distroless runtime에 포함하고, 공통 Protobuf와 Android 사본의 drift를 CI에서 차단했습니다. 최종 봉인본에서는 클라우드 결합을 정리했습니다.

제품 적용 판단과 코드 범위

ML1은 독립 산책에서 제품 가치를 충분히 확인하지 못했고 ML2는 독립 사건 표본이 부족했습니다. 상대 위치도 리드줄 규모의 정확도를 만족하지 못했습니다. 관측·사용자 확인 기록·수집 복구 코드는 남기고 검증되지 않은 자동 판정을 차단했습니다.

링크는 **비공개 원본의 정확한 구간 발췌**입니다. 원본 커밋·경로·줄·해시는 코드 근거 목록에 기록했습니다. 기존 설계·평가 사례집도 함께 읽을 수 있습니다. 최근 구현·시험에는 AI 에이전트를 활용했고 문제·현장 데이터·평가 기준·채택과 중단 판단은 직접 맡았습니다.